OPERATING SYSTEMS

ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks

You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation

You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating
- Explain position-independent code

Experimental Investigation

You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks

You must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

Evaluation

You must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation

You must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>


#define PAGE_SIZE 1024

#define PAGES 8


/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};


int main(void)

{

unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049}; for
(int i = 0; i < 6; i++) {

unsigned a = logical[i];
```

Expected output:

```
0 -> page 0 offset          0 : physical 5120

1500 -> page 1 offset  476 : physical 2524

3100 -> page 3 offset     28 : physical 7196
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks

You must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation

You must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation

You must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes
- Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks

You must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation

You must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation

You must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks

You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation

You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it
- Compare the algorithms for a workload with strong locality

Experimental Investigation

You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

Which page-replacement policy would you recommend for the web server? Use your measurements.

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
for (int i = 0; i < n; i++) if (f[i] == page) return i; return -
1;
}


int fifo(const int ref[], int len, int frames)
{
int f[MAXF], used = 0, next = 0, faults = 0; for
(int i = 0; i < len; i++) {
if (find(f, used, ref[i]) >= 0) continue; faults++;
if (used < frames) f[used++] = ref[i];
else { f[next] = ref[i]; next = (next + 1) % frames; }
}
return faults;
}


int lru(const int ref[], int len, int frames)
{
int f[MAXF], last[MAXF], used = 0, faults = 0; for
(int i = 0; i < len; i++) {
int k = find(f, used, ref[i]);
if (k >= 0) { last[k] = i; continue; } faults++;
// TODO 1: If a frame is free use it; otherwise replace the frame whose last use
//             (last[]) is the oldest. Record last[] = i for the frame used


}
return faults;
}


int optimal(const int ref[], int len, int frames)
{
int f[MAXF], used = 0, faults = 0; for
(int i = 0; i < len; i++) {
if (find(f, used, ref[i]) >= 0) continue; faults++;
if (used < frames) { f[used++] = ref[i]; continue; } int victim
= 0, farthest = -1;
for (int j = 0; j < frames; j++) {
int next = len;                                  /* never used again */
for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; } if (next >
farthest) { farthest = next; victim = j; }
}
f[victim] = ref[i];
}
return faults;
}


int main(void)
{
int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
int len = 20;
for (int fr = 3; fr <= 4; fr++)
printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr), lru(ref,
len, fr), optimal(ref, len, fr));
int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n", fifo(b, 12,
3), fifo(b, 12, 4));
return 0;
}
```

Expected output:

```
 3 frames: FIFO 15, LRU 12, Optimal 9
 4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management **4 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks

You must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation

You must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation

You must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles **10 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock())
- Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator
- Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

OPERATING SYSTEMS – ASSIGNMENT 3 (CO3)

Q1. Linking, Loading and Address Binding

1. Introduction

Linking and loading are important memory-management activities. Linking combines program modules and libraries, while loading places the executable program into memory for execution. A banking application may contain several modules, such as customer details, account management, and encryption. If the encryption library is updated, dynamic linking can avoid distributing a completely new executable to every branch.

2. Linking

Linking is the process of combining object files and required libraries to create an executable program.

There are two main types.

Static Linking

Required library code is copied into the executable.
The executable can run without the original library file being installed separately.

The executable is generally larger.
Updating a library usually requires rebuilding or replacing the executable.

Dynamic Linking

The executable refers to shared libraries.
The shared library is loaded when the program starts or when it is required.

The executable is generally smaller.
Multiple programs can share library code in memory.
Updating a compatible shared library may not require rebuilding the executable.

3. Static and Dynamic Linking Comparison

Feature Static linking Dynamic linking

In practice, the two activities are closely related but are performed by different components of the system. The linker resolves references between separately compiled modules, while the loader prepares the executable image and transfers control to its entry point. Understanding these stages helps explain how a source program becomes a running process.

These mechanisms also affect portability, memory use, startup time, and software maintenance. A well-designed application chooses a linking method according to whether libraries change frequently, whether multiple programs need to share code, and whether the target environment can guarantee the required dependencies.

Executable

Larger Smaller

size

Library code Included in executable Stored in shared library

Memory

May duplicate library code Code pages can be shared

usage

Updating Rebuild or replace executable Replace compatible library

Fewer runtime library

Dependency Requires shared library

dependencies

Can avoid dynamic-library May require library loading and

Startup

loading work relocation

4. Address Binding

Address binding connects program instructions and data to memory

addresses.

Compile-Time Binding

Used when the memory location is known during compilation.
The compiler generates address-dependent code.
If the location changes, recompilation may be needed.
Load-Time Binding

The final memory location is decided when the program is loaded.
The loader adjusts addresses before execution.
The program cannot be moved freely during execution without further support.

Execution-Time Binding

Address translation occurs while the program runs.
The process can be moved between memory locations.
Hardware support such as the memory management unit (MMU) is required.

5. Linker and Loader

Linker Loader

Combines object files Loads executable into memory

The choice between static and dynamic linking is therefore a trade-off rather than an absolute rule. Static linking can be convenient for deployment in a controlled environment because dependencies are bundled. Dynamic linking is often preferable on general-purpose systems where libraries are maintained centrally and shared by many applications.

Compatibility must still be checked. A library update should preserve the expected exported symbols, calling conventions, and data structures. An incompatible replacement can cause a program to fail at startup or behave incorrectly even if the library file is present.

Resolves external symbols Creates the process memory image

Connects library references Maps program segments

Produces executable output Starts program execution

6. Position-Independent Code

Position-independent code (PIC) can execute correctly regardless of where it

is loaded in memory.

It is commonly used in shared libraries because the same library can be

mapped at different virtual addresses in different processes.

7. Static Linking Example

```
#include <stdio.h>
int main()
{
printf("Banking Application\n");
return 0;
}
```

8. Dynamic Linking Example

gcc bank.c -o bank_dynamic

9. Runtime Library Loading

The dlopen() function loads a shared library at runtime. The dlsym() function

obtains the address of a function from that library.

Example program:

```
#include <stdio.h>
#include <dlfcn.h>
int main()
{
void *handle;
void (*hello)();
```

The operating system and loader cooperate to establish a valid process image. The loader maps executable segments, arranges stack and heap regions, applies relocations when necessary, and prepares arguments and environment variables. Address binding determines when symbolic or logical references become concrete memory addresses.

Position-independent code is especially useful for shared libraries because the same code can be mapped at different virtual addresses in different processes. This supports address-space layout randomization and reduces the need to duplicate library code.

```
    handle = dlopen("./libhello.so", RTLD_LAZY);
    if (!handle) {
    printf("Library loading failed\n");
return 1;
    }
    hello = (void (*)())dlsym(handle, "hello");
    if (!hello) {
    printf("Function not found\n");
dlclose(handle);
return 1;
    }
    hello();
dlclose(handle);
    return 0;
    }
```

Output:

Hello from shared library

10. Address Observation

The following Linux commands help inspect memory mappings:

cat /proc/self/maps

11. Experimental Investigation

The following experiments can be performed:

1. Compare the sizes of static and dynamic executables.

2. Use ldd to list dynamic dependencies.

The example demonstrates the main steps of runtime linking: opening the shared object, locating an exported function, checking for errors, invoking the function, and releasing the handle. Each operation should be checked because a missing file or symbol is a normal runtime failure that must be handled safely.

When compiling a program that uses this interface, the appropriate system headers and compiler options are required. The program should also be tested with a valid library and with a missing or incompatible library so that both successful and failure paths are observed.

3. Replace a compatible shared library and test the application again.

4. Run two program instances and inspect their memory mappings.

5. Rename or temporarily remove a required library in a safe test

environment and observe the loading error.

Actual start-up times and memory usage should be measured on the Linux

system. They should not be assumed.

12. Alternative: Runtime Loading with dlopen()

Instead of loading the encryption library when the program starts, the

application can call dlopen() only when encryption is required.

Advantages:

Reduces library memory use when encryption is not needed.
Allows optional features.
Keeps the initial program loading process simpler when the library is not required.

Disadvantages:

The first encryption request may take longer.
The program must handle missing libraries.
Function lookup errors must be handled.
Library compatibility must be checked.

13. Final Recommendation

The bank should use dynamic linking for common libraries that are shared by

multiple applications. It can use runtime loading for optional encryption

features that are not required in every operation.

Testing should be performed in a controlled directory so that system libraries are not accidentally removed or overwritten. The memory map of each process can be inspected to verify whether the shared library is mapped and whether code pages are shared.

Record the observed output, library path, compiler command, and system architecture. Results may differ across operating systems and loader implementations, so the experiment should describe the actual environment instead of assuming a universal output.

Q2. Paging

1. Introduction

Paging is a memory-management technique that divides a process's logical memory into fixed-size pages and physical memory into fixed-size frames. Each page can be placed in any available frame. A page table stores the relationship between logical pages and physical frames.

Paging is useful for a college server that runs many programs of different sizes.

2. External Fragmentation

External fragmentation occurs when free memory is divided into small, separate blocks. A process may not find one sufficiently large contiguous block even when the total free memory is sufficient.

Example:

Physical Memory

+---------+---------+---------+---------+

| Process | Free 2MB| Process | Free 3MB|

+---------+---------+---------+---------+

Total free memory = 5 MB

3. How Paging Works

1. The CPU generates a logical address.

2. The logical address is divided into a page number and an offset.

3. The page number is used to access the page table.

4. The page table provides the corresponding frame number.

5. The frame number and offset form the physical address.

\[Logical\ Address = Page\ Number + Offset \]

\[Physical\ Address = Frame\ Number \times Page\ Size + Offset \]

4. Address Translation Example

Given:

Page size = 1024 bytes
Logical address = 1500

Paging makes allocation flexible because a process does not need one continuous block of physical memory. The page table provides the mapping between virtual page numbers and physical frame numbers, while the offset remains unchanged during translation.

Page size is selected by hardware and operating-system design. A small page size can reduce internal fragmentation but increases the number of page-table entries. A larger page size reduces table overhead but may waste memory when allocations are small.

Page 1 is stored in frame 2
Page number:

$$ p = 1500/1024 = 1 $$

Offset:

$$ d = 1500 \bmod 1024 = 476 $$

Physical address:

$$ PA = 2 \times 1024 + 476 $$

$$ PA = 2524 $$

5. Page Table Size

For a 32-bit address space and 4 KB pages:

$$ 4KB=2^{12}\ bytes $$

Number of pages:

$$ \frac{2^{32}}{2^{12}}=2^{20} $$

Therefore, the page table requires 2^{20} entries for a single-level page table covering the full address space.

For a 48-bit address space:

$$ \frac{2^{48}}{2^{12}}=2^{36} $$

Therefore, a single-level page table would require 2^{36} entries.

If each entry occupies 4 bytes:

Address space Number of entries Page-table size

32-bit 2^{20} 4 MB

48-bit 2^{36} 256 GB

These are theoretical single-level table sizes. Real systems commonly use hierarchical or other page-table designs to avoid allocating such enormous tables.

6. Effective Access Time

A Translation Lookaside Buffer (TLB) is a small, fast cache that stores recent page-table translations.

Assume:

Memory access time = 100 ns
The address translation is performed by separating the logical address into page number and offset. The page number indexes the page table; the frame number found there is combined with the same offset to obtain the physical address. If the page-table entry is invalid, the reference is not translated as an ordinary resident access.

Page-table size grows with the virtual address space and inversely with page size. Multi-level paging avoids allocating a large contiguous page table for unused portions of an address space.

TLB lookup time = 10 ns
TLB hit ratio = 95%
On a TLB hit:

\[10+100=110ns \]

On a TLB miss, assuming one additional memory access for a single-level

page table:

\[10+100+100=210ns \]

Effective access time:

\[EAT=0.95(110)+0.05(210) \]

\[EAT=115ns \]

Without a TLB, a single-level page-table lookup generally requires two memory

accesses:

\[EAT=100+100=200ns \]

7. Page Size Comparison

Page

Advantages Disadvantages

size

Less internal fragmentation per

1 KB More page-table entries

allocation

4 KB Balanced size for many workloads Moderate table overhead

Fewer page-table entries and useful Can waste more memory for

2 MB

for large memory regions small allocations

8. Types of Page Tables

Hierarchical Page Table

Divides the page table into multiple levels.
Allocates lower-level tables only when required.
Reduces memory consumption for sparse address spaces.
Hashed Page Table

Uses a hash function to locate page-table entries.
Useful for large address spaces.
Collisions must be handled.
The translation lookaside buffer (TLB) is a small associative cache of recent page-table translations. Its purpose is to avoid an additional memory reference for every address translation. The hit ratio has a major effect on effective access time, particularly when page-table walks require multiple memory accesses.

Actual systems may use multi-level TLBs and hardware page walkers. The simplified calculation assumes one memory access for data and one extra access on a miss, so the assumptions should be stated clearly in numerical answers.

Inverted Page Table

Maintains one entry per physical frame rather than one entry for every virtual page.

Reduces table size.
Translation may require searching or hashing.

9. Protection and Shared Pages

Paging supports memory protection using page-table permission bits.

Examples:

Read permission.
Write permission.
Execute permission.
User and kernel access permissions.
Shared pages allow multiple processes to map the same physical memory.

This is useful for shared libraries and read-only program code.

10. Internal Fragmentation

Internal fragmentation is unused space inside an allocated page.

\[Internal\ Fragmentation = Allocated\ Page\ Space - Required\ Space \]

For example, if a process needs 5,000 bytes and the page size is 4,096 bytes:

Two pages are required.

\[Allocated=8192\ bytes \]

\[Fragmentation=8192-5000=3192\ bytes \]

11. Page Translation C Program

```
    #include <stdio.h>
```

#define PAGE_SIZE 1024

#define PAGES 8

```
    int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};
    int main()
```

Protection bits commonly include read, write, and execute permissions. A process should not be able to modify kernel pages or execute data pages unless the policy permits it. Shared pages can be mapped into more than one address space, but each process still observes its own protection rules.

Inverted and hashed page tables are alternative designs used to reduce memory overhead for large address spaces. Their lookup mechanisms differ from the conventional per-process page table, and performance depends on the hardware and workload.

```
    {
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
    unsigned a = logical[i];
    unsigned p = a / PAGE_SIZE;
    unsigned d = a % PAGE_SIZE;
    printf("%5u -> page %u offset %6u : ", a, p, d);
    if (p >= PAGES)
    printf("invalid address\n");
    else if (page_table[p] == -1)
    printf("page fault\n");
    else {
    unsigned physical =
```
page_table[p] * PAGE_SIZE + d;
```
    printf("physical %u\n", physical);
    }
    }
    return 0;
    }
```
Expected output:

0 -> page 0 offset 0 : physical 5120

A translation program is useful for checking boundary conditions: the first byte of a page, the last byte of a page, an address exactly at the end of the address space, and an address whose page is not currently resident. These cases help detect arithmetic errors.

Unsigned arithmetic and page-size constants should be chosen carefully in C. A production implementation would also validate input and avoid assuming that every logical address is legal.

1500 -> page 1 offset 476 : physical 2524

3100 -> page 3 offset 28 : physical 7196

6200 -> page 6 offset 56 : physical 3128

9000 -> page 8 offset 808 : invalid address

2049 -> page 2 offset 1 : page fault

12. Alternative: Inverted Page Table

An inverted page table maintains one entry for every physical frame.

Advantages:

Table size depends on physical memory.
Reduces memory used by page tables.
Suitable for large virtual address spaces.
Disadvantages:

Address translation can require a hash lookup.
Hash collisions increase search work.
Shared memory and process identification require additional handling.

13. Final Recommendation

For a multi-user college server, 4 KB pages provide a practical balance

between memory management and fragmentation. Large pages can be used

```
for selected workloads that benefit from fewer translation entries.
```

The output must be interpreted using the page table and the selected frame allocation. A valid page may be resident, absent, or outside the process's permitted virtual range. These conditions are different and should not be reported as the same error.

An inverted page table stores information indexed by physical frame. It can reduce table memory, but finding the matching process and virtual page may require hashing or searching. Thus, lower space overhead can introduce additional translation complexity.

Q3. Segmentation

1. Introduction

Segmentation is a memory-management technique that divides a program into

logical parts called segments.

Examples include:

Code segment
Data segment
Stack segment
Heap segment
Each segment has a different purpose and may have a different size.

2. Segment Table

A segment table stores the base address and limit of each segment.

Base: Starting physical address of the segment.
Limit: Length or maximum valid offset of the segment.

3. Address Translation

A logical address contains:

\[(Segment\ Number,\ Offset) \]

The operating system or hardware checks whether the offset is within the

segment limit.

If:

\[Offset < Limit \]

then:

\[Physical\ Address = Base + Offset \]

Otherwise, an address-protection error occurs.

4. Example

Segment Base Limit

Code 1000 500

Data 3000 800

Stack 5000 400

Segments reflect the logical structure of a program rather than dividing memory into equal-sized pieces. A segment table stores a base address and a limit for each segment, and the logical address identifies both the segment number and its offset.

Segmentation supports meaningful protection and sharing. For example, code can be marked read-only and shared, while data and stack remain private and writable. Its principal drawback is external fragmentation because segments have variable sizes.

For Data segment offset 200:

\[PA=3000+200=3200 \]

For Data segment offset 900:

Since 900 is greater than the limit of 800, the address is invalid.

5. Segmentation Program

```
    #include <stdio.h>
    int main()
    {
    int base[] = {1000, 3000, 5000};
    int limit[] = {500, 800, 400};
    int segment, offset;
    printf("Enter segment number: ");
scanf("%d", &segment);
    printf("Enter offset: ");
scanf("%d", &offset);
    if (segment < 0 || segment >= 3)
    printf("Invalid segment\n");
    else if (offset < 0 || offset >= limit[segment])
    printf("Segmentation fault\n");
    else
    printf("Physical address = %d\n",
base[segment] + offset);
```

A valid reference must satisfy two checks: the segment number must identify an existing segment, and the offset must be less than that segment's limit. Only then is the base added to the offset. This validation prevents one segment from accessing another segment's memory accidentally.

The C example models the translation process using arrays for base and limit values. It is a conceptual simulation and does not replace hardware memory protection; the operating system and processor enforce protection in real systems.

```
return 0;
}
```

6. Segmentation with Paging

Segmentation with paging combines the benefits of both techniques.

1. A program is divided into logical segments.

2. Each segment is divided into pages.

3. Each segment has a page table.

4. The page table maps pages to physical frames.

5. The physical address is generated using the frame number and offset.

This approach supports logical program organisation while avoiding the need

```
for each segment to occupy one continuous physical-memory block.
```

7. Protection and Sharing

Segmentation supports protection by assigning different permissions to

different segments.

For example:

Code segment: Read and execute.
Data segment: Read and write.
Stack segment: Read and write.
Code segments can be shared among processes running the same program,

provided that they are mapped with suitable permissions.

8. External Fragmentation

Segmentation uses variable-sized memory blocks. Over time, free memory

may become divided into small gaps.

This produces external fragmentation.

Compaction may combine scattered free spaces, but it can be expensive

because memory contents may need to be moved.

9. Comparison of Memory Schemes

Feature Pure paging Pure segmentation Segmented paging

Segmentation with paging preserves a program's logical segments while avoiding the need to allocate each entire segment contiguously. Each segment has its own page table, and a logical address is translated using the segment number, page number, and offset.

The combined design increases translation steps and table-management complexity. It can nevertheless provide useful protection boundaries and reduce external fragmentation compared with pure segmentation.

Fixed-size Segments divided

Division Variable-size segments

pages into pages

External No at physical page

No Yes

fragmentation allocation level

Internal Usually limited to

Possible Possible

fragmentation allocation overhead

Logical

Less direct Strong Strong

organisation

Page

Protection Segment permissions Both levels

permissions

Segment and page-

Sharing Page-based Segment-based

based

10. Linux Memory Mapping

Linux represents a process's memory regions through /proc.

Use:

cat /proc/PID/maps

The output may show:

Executable code
Shared libraries
Heap
Stack
Memory-mapped files

Modern Linux primarily uses paging for virtual-memory management. The

memory-map regions reflect logical areas of a process, rather than a

traditional pure segmentation scheme.

11. Segmentation Fault

A segmentation fault occurs when a process accesses memory in an invalid or

unauthorised way.

Fragmentation is an important criterion when comparing memory-management approaches. Internal fragmentation is unused space inside an allocated unit; external fragmentation is free space split into separated holes that cannot satisfy a large contiguous request.

Paging removes external fragmentation at the physical-frame allocation level, but it does not remove all overhead. Page tables consume memory, and the final page of a process may contain unused bytes.

For example, accessing memory outside an allocated array or writing to read-only memory can cause a segmentation fault.

The operating system detects the invalid access and normally sends the process a SIGSEGV signal.

12. Alternative: Pure Paging

If the system used pure paging instead of segmentation:

All memory would be divided into fixed-size pages.
Logical addresses would be translated using page tables.
External fragmentation would be avoided.
Internal fragmentation could remain.
Protection and sharing would be implemented through page-table permissions.

A segmentation fault is an observable symptom of an invalid memory reference, not a detailed diagnosis by itself. The cause may be a null pointer, an out-of-bounds array access, use-after-free, or an attempt to write to protected memory.

Debugging tools and compiler checks can help identify the exact source of the invalid reference. The operating system's protection mechanism prevents the faulty process from corrupting unrelated processes or kernel memory.

Q4. Virtual Memory and Demand Paging

1. Introduction

Virtual memory allows a process to use an address space larger than the available physical RAM.

Demand paging loads a page into RAM only when the process actually accesses it.

A medical-imaging program requiring 24 GB of memory can run on a computer with 16 GB of RAM if its active working set fits in memory and sufficient backing storage is available.

2. Demand Paging

In demand paging, pages are loaded into physical memory when needed.

Advantages:

Reduces initial memory loading.
Allows large programs to run with limited RAM.
Avoids loading unused pages.
Improves memory utilisation.

3. Page Fault Handling

A page fault occurs when a process accesses a page that is not currently available in physical memory.

Steps:

1. The CPU generates a memory address.

2. The MMU detects that the page is not present.

3. A page-fault exception occurs.

4. The operating system checks whether the memory access is valid.

5. The OS finds a free frame or selects a page for replacement.

6. The required page is loaded from storage.

7. The page table is updated.

8. The interrupted instruction is restarted.

4. Effective Access Time

Given:

Memory access time = 100 ns

Virtual memory separates the addresses used by a program from the physical locations available in RAM. Only the actively needed pages must be resident, which allows more processes to run and can make large programs practical even when their total virtual size exceeds physical memory.

Demand paging is beneficial when locality exists: during a time interval, a process tends to access a limited set of pages. If the working set exceeds available frames, frequent page faults may cause thrashing and reduce throughput.

Page-fault service time = 8 ms
Convert:

$$ 8ms=8,000,000ns $$

Formula:

$$ EAT=(1-p)\times Memory\ Time+ p\times Page\ Fault\ Time $$

where p is the page-fault rate.

Case 1: One fault in 1,000 accesses

$$ p=0.001 $$

$$ EAT=0.999(100)+0.001(8000000) $$

$$ EAT=8099.9ns $$

Case 2: One fault in 100,000 accesses

$$ p=0.00001 $$

$$ EAT=0.99999(100)+0.00001(8000000) $$

$$ EAT=179.999ns $$

Case 3: One fault in 1,000,000 accesses

$$ p=0.000001 $$

$$ EAT=0.999999(100)+0.000001(8000000) $$

$$ EAT=107.9999ns $$

5. Effective Access Time Table

Page-fault rate Effective access time

1 in 1,000 8,099.9 ns

1 in 100,000 179.999 ns

1 in 1,000,000 107.9999 ns

The results show that even a small page-fault rate can significantly increase

memory-access time because storage access is much slower than RAM

access.

6. Copy-on-Write

Copy-on-Write (COW) is a technique used when a process is duplicated.

The page-fault rate must be interpreted as a probability per memory reference. Because page-fault service time is measured in milliseconds while ordinary memory access is measured in nanoseconds, even a small fault probability can dominate effective access time.

Consequently, systems try to keep frequently used pages resident and reduce avoidable faults. The calculation is an expectation; it does not mean every access takes the same time.

Instead of immediately copying every memory page, the parent and child initially share the same physical pages.

When one process attempts to modify a shared page:

1. A protection fault occurs.

2. The OS creates a private copy of the page.

3. The process modifies its own copy.

This reduces the cost of fork() when the child does not modify most of the parent's memory.

7. Thrashing

Thrashing occurs when the system spends too much time moving pages between RAM and storage rather than executing processes.

Causes:

Insufficient physical memory.
Too many active processes.
Poor locality of reference.
Insufficient frames allocated to processes.
Effects:

High page-fault rate.
Increased disk activity.
Low CPU utilisation.
Slow application response.

8. Working-Set Model

The working set is the group of pages actively used by a process during a recent time window.

The operating system can use working-set information to estimate how many frames a process requires.

If the total working-set demand exceeds available physical frames, the OS may reduce the number of active processes or adjust memory allocation.

9. Frame Allocation

Equal Allocation

Each process receives the same number of frames.

Copy-on-write improves efficiency when a child process immediately executes another program after creation. Copying every page at process creation would waste time and memory, so pages are shared until a write makes a private copy necessary.

The mechanism requires page-level protection and fault handling. The kernel must update mappings consistently and preserve the original contents for the process that still uses the shared page.

Proportional Allocation

Processes receive frames based on their memory requirements or size.

Method Advantage Disadvantage

Large processes may need

Equal allocation Simple

more frames

Proportional Better matches memory

Requires tracking process sizes

allocation needs

10. Experimental Investigation

The following experiments can be performed:

getconf PAGESIZE

11. Alternative: 32 GB RAM

If the laptop has 32 GB RAM, the 24 GB application can potentially keep most

of its working data in physical memory.

Compared with a 16 GB machine:

Fewer page faults may occur.
Storage access may be reduced.
Application response may improve.
The system has more room for other processes.
However, the benefit depends on the actual working set and memory usage.

12. Final Recommendation

The radiologist should first improve the program's memory locality and

remove unnecessary memory usage. If the application consistently requires a

large working set, increasing RAM to 32 GB is a practical solution.

Frame allocation is also affected by whether the system uses local or global replacement. Local replacement limits a process to its own frames, while global replacement allows it to take frames from other processes. Global policies can improve overall utilisation but may cause one process to affect another.

Allocation policies should be evaluated with workload information, not only process size. A process's active working set and its phase of execution can change over time.

Q5. Page-Replacement Techniques

1. Introduction

Page replacement is used when a process requires a page but no free physical frame is available.

The operating system selects an existing page to remove and loads the required page.

A web server frequently accesses stored data pages. A suitable page-replacement algorithm helps reduce page faults and improve performance.

2. FIFO Page Replacement

FIFO stands for First In First Out.

The oldest loaded page is replaced first.
It is simple to implement.
It does not consider how frequently or recently a page is used.
It may suffer from Belady's anomaly.

3. LRU Page Replacement

LRU stands for Least Recently Used.

Replaces the page that has not been used for the longest time.
Uses recent access history.
Performs well when recent usage predicts future usage.
Does not suffer from Belady's anomaly.

4. Optimal Page Replacement

Optimal replacement removes the page whose next use is farthest in the future.

Produces the minimum possible page faults for a known reference string.

Requires knowledge of future page references.
Is mainly used as a theoretical comparison.

5. Reference String

The assignment provides:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3,

0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Page replacement is invoked only when the required page is not resident and no suitable free frame is available. The selected victim may be written to backing store if it has been modified; clean pages can often be discarded and read again when needed.

The algorithm affects fault rate, response time, disk traffic, and fairness. A good policy tries to retain pages that will be used soon while avoiding excessive bookkeeping overhead.

6. Page Fault Results

Frames FIFO LRU Optimal

3 15 12 9

4 10 8 8

The results show that LRU performs better than FIFO for this reference string.

Optimal produces the minimum number of page faults.

7. LRU C Program

```
    #include <stdio.h>
```

#define MAXF 8

```
    int find(const int f[], int n, int page)
    {
    for (int i = 0; i < n; i++)
    if (f[i] == page)
```

return i;

return -1;

```
    }
    int lru(const int ref[], int len, int frames)
    {
    int f[MAXF], last[MAXF];
    int used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
    int k = find(f, used, ref[i]);
    if (k >= 0) {
```

The reference string is a controlled way to compare algorithms because every algorithm receives the same sequence of page references and frame count. A page fault occurs when the referenced page is not in any available frame; a hit does not require replacement.

Optimal replacement is a theoretical benchmark because it requires knowledge of future references. LRU uses past activity as an approximation, while FIFO uses arrival order and can evict a frequently used page.

```
last[k] = i;
continue;
    }
faults++;
    if (used < frames) {
f[used] = ref[i];
last[used] = i;
used++;
} else {
    int victim = 0;
    for (int j = 1; j < frames; j++)
    if (last[j] < last[victim])
victim = j;
f[victim] = ref[i];
last[victim] = i;
    }
    }
return faults;
    }
    int main()
    {
    int ref[] = {
7, 0, 1, 2, 0, 3, 0, 4, 2, 3,
```

The LRU implementation records the most recent index at which each frame's page was used. On a miss, the page with the smallest last-used index is selected as the least recently used. The state must be updated after both hits and insertions.

Care is needed when a frame is empty and when the reference string contains repeated values. Testing small strings manually is an effective way to verify the program before running larger experiments.

0, 3, 2, 1, 2, 0, 1, 7, 0, 1

};

```
    int len = 20;
    printf("LRU 3 frames = %d\n", lru(ref, len, 3));
    printf("LRU 4 frames = %d\n", lru(ref, len, 4));
    return 0;
    }
```

8. Belady's Anomaly

Belady's anomaly occurs when increasing the number of frames increases the

number of page faults.

The assignment gives the reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO results:

Frames Page faults

3 9

4 10

The number of faults increases from 9 to 10 even though the number of frames

increases.

This is Belady's anomaly.

9. Why LRU Does Not Have Belady's Anomaly

LRU is a stack algorithm.

The pages held with \(n\) frames form a subset of the pages held with \(n+1\)

frames for the same reference history.

Therefore, increasing the number of frames cannot increase the number of

page faults under LRU.

10. Clock Algorithm

The Clock algorithm is also called Second-Chance replacement.

The main program supplies a reference string and invokes the algorithm with different frame counts. The output should be compared with a hand-worked trace to confirm correctness. The exact fault totals depend on the reference sequence and the number of frames.

Belady's anomaly is associated with some non-stack algorithms such as FIFO. Stack algorithms, including LRU and Optimal, do not exhibit the anomaly because the set of pages held with fewer frames is contained in the set held with more frames.

Each page has a reference bit.

Working:

1. The algorithm checks the page pointed to by the clock hand.

2. If its reference bit is zero, the page can be replaced.

3. If its reference bit is one, the bit is cleared and the page receives a second chance.

4. The clock hand moves to the next page.

5. The process repeats until a replacement candidate is found.

11. Comparison of Algorithms

Feature FIFO LRU Optimal Clock

Least

Replacement Oldest Farthest future

recently Reference bit

basis loaded use

used

More Not practical for

Implementation Simple Moderate

complex future knowledge

Belady's Not guaranteed to

Possible No No

anomaly be absent

Depends Good for Practical

Performance Best theoretical

on order locality approximation

12. Complexity Analysis

For a simple implementation using a linear search through frames:

Searching for a page takes \(O(f)\), where \(f\) is the number of frames.
LRU victim selection takes \(O(f)\).
Processing \(n\) references takes \(O(nf)\).
More advanced implementations can use hash tables and linked lists to improve lookup and replacement efficiency.

13. Experimental Investigation

The assignment asks for:

Strong-locality reference strings.
Random reference strings.
Frame counts from 3 to 10.

Clock replacement is a practical approximation to LRU. It avoids maintaining a fully ordered list after every access and instead uses a circular pointer and reference bits. Pages that have recently been accessed receive a second chance before eviction.

The algorithm's effectiveness depends on the workload and the number of frames. It is widely used because it balances modest overhead with reasonable replacement behaviour.

Running-time measurements for 100,000 references.
A measurement table can be maintained as follows:

Frames FIFO faults LRU faults Optimal faults

3 Record result Record result Record result

4 Record result Record result Record result

5 Record result Record result Record result

6 Record result Record result Record result

7 Record result Record result Record result

8 Record result Record result Record result

9 Record result Record result Record result

10 Record result Record result Record result

Actual experimental values must be collected by executing the algorithms.

14. Alternative: Clock Instead of Exact LRU

The Clock algorithm uses a reference bit rather than maintaining exact access

timestamps or a complete recency list.

Advantages:

Lower bookkeeping overhead.
Requires approximately one reference bit per frame.
Suitable for practical operating-system implementations.
Disadvantages:

It only approximates LRU.
It may replace a page that was used more recently than another page.
Its page-fault count can be higher than exact LRU.

15. Final Recommendation

For a web server, LRU or a Clock-based approximation is suitable. LRU

performs well for workloads with strong locality, while Clock reduces the

overhead of maintaining exact usage order.

For a fair experiment, use the same reference string, record the frame count, and repeat each algorithm under identical conditions. Include the compiler version and machine details when timing the program, because runtime measurements vary with system load.

Fault count and execution time are different metrics. An algorithm can produce fewer faults but require more CPU time for bookkeeping; both should be considered when interpreting results.

Q6. Disk Management

1. Introduction

Disk management is the process of organising, allocating, and maintaining secondary storage devices.

A hospital records server requires reliable storage because patient records must remain available even if a disk fails.

2. Low-Level Formatting

Low-level formatting prepares a disk into sectors that can store data.

It organises the storage device into addressable units used by the disk controller.

3. Partitioning

Partitioning divides a physical disk into separate logical regions.

For example:

System partition
Data partition
Backup partition
Swap partition

4. Logical Formatting

Logical formatting creates a file system on a partition.

Examples include:

ext4

XFS

NTFS

FAT32

The file system organises files, directories, metadata, and free space.

5. Boot Block and Boot Process

The boot block contains information or code required to begin the boot process.

Boot process:

1. The computer starts.

Secondary storage is persistent, unlike RAM, and is organised into partitions, file systems, and volumes. Disk management includes initialisation, formatting, capacity planning, monitoring health, and recovery from failure.

For a server containing important records, reliability and recovery are as important as raw capacity. A design should account for backups, access controls, monitoring, and tested restoration procedures.

2. BIOS or UEFI performs initial hardware checks.

3. The firmware selects a boot device.

4. A bootloader is started.

5. The bootloader loads the operating-system kernel.

6. The kernel initialises memory, devices, and system services.

7. The operating system becomes ready for use.

6. Bad Block Handling

A bad block is a storage sector that cannot reliably store or retrieve data.

The operating system and storage device may handle bad blocks using:

Bad-block detection.
Sector remapping.
Spare sectors.
File-system checks.
Backup and recovery.
Regular monitoring helps identify storage problems before they cause serious

data loss.

7. Swap Space

Swap space is storage used as an extension of RAM.

It may be implemented as:

A dedicated swap partition.
A swap file.
A swap partition provides a dedicated area, while a swap file is easier to

create, resize, and manage.

For a hospital server, sufficient physical RAM and reliable storage should be

prioritised. Swap should not be treated as a replacement for adequate RAM.

8. RAID Levels

RAID stands for Redundant Array of Independent Disks.

RAID Minimum

Method Fault tolerance

level disks

RAID 0 Striping 2 No disk-failure protection

The boot process is a chain of stages in which each component loads or initialises the next. Firmware settings determine the boot mode and device order, while the bootloader locates and loads the kernel and possibly an initial RAM filesystem.

Problems at different stages have different symptoms. A missing boot device points to firmware or storage configuration; a kernel panic occurs later, after the kernel has begun executing.

Can survive a disk failure in a two-

RAID 1 Mirroring 2

disk mirror

Striping with

RAID 5 3 Can survive one disk failure

distributed parity

RAID Mirroring and Can survive multiple failures if they

4

10 striping do not destroy a mirror pair

9. Capacity of Six 4 TB Disks

Total raw capacity:

\[6 \times 4 = 24TB \]

RAID level Usable capacity

RAID 0 24 TB

RAID 1 12 TB for three mirrored pairs

RAID 5 20 TB

RAID 10 12 TB

These values assume equal-sized disks and decimal TB units. RAID 1 capacity

depends on the mirror layout; three mirrored pairs provide 12 TB.

10. RAID Comparison

RAID 0

High performance.
Uses all disk capacity.
No redundancy.
Failure of one disk can destroy the array.

RAID 1

Stores duplicate data.
Provides strong redundancy.
Uses approximately half the raw capacity in mirrored pairs.

RAID 5

Uses distributed parity.
Provides good capacity efficiency.
RAID level selection should consider fault tolerance, write performance, usable capacity, and rebuild behaviour. RAID is not a substitute for backup: deletion, corruption, ransomware, and site-level disasters can affect all members of an array.

Hot spares and monitoring can reduce the time during which an array is degraded. Rebuild time depends on disk capacity, controller performance, and workload.

Can survive one disk failure.
Rebuilding after failure can place significant stress on remaining disks.

RAID 10

Combines striping and mirroring.
Provides good read and write performance.
Offers redundancy.
Uses half the raw capacity.

11. Swap Partition vs Swap File

Feature Swap partition Swap file

Location Dedicated partition File within a file system

Setup Requires partitioning Easier to create

Resizing More difficult Usually easier

Management Separate storage area Managed like a file

Flexibility Lower Higher

12. Linux Disk Commands

Display block devices:

Lsblk

Display memory and swap:

free -h

13. Alternative: RAID 6

RAID 6 uses two independent parity blocks.

For six disks of 4 TB:

\[Usable\ Capacity=(6-2)\times4 \]

\[=16TB \]

Advantages:

Can survive two disk failures.
Provides stronger fault tolerance than RAID 5.
Suitable for large storage systems where rebuilding takes time.
RAID 10 mirrors pairs and stripes data across those pairs. It offers strong random I/O performance and straightforward rebuilds, but only about half of raw capacity is usable. Failure tolerance depends on which disks fail; multiple failures can be survived if they occur in different mirror pairs.

RAID 5 and RAID 6 use distributed parity to improve capacity efficiency. RAID 6 can tolerate two disk failures, while RAID 5 can tolerate one, but parity updates introduce additional write work.

Disadvantages:

Less usable capacity.
More parity calculations.
Write performance can be lower than RAID 5 or RAID 10.

14. Final Recommendation

For a hospital records server, RAID 10 or RAID 6 is suitable depending on

performance and capacity requirements.

RAID 10 is appropriate when high performance and fast recovery are
important.

RAID 6 is appropriate when surviving two disk failures is a priority.
RAID must be combined with independent backups because RAID alone does

not protect against accidental deletion, malware, or site-level disasters.

For a records server, the recommendation should be tied to the required recovery point, recovery time, capacity growth, and budget. RAID 10 is a strong performance-oriented option; RAID 6 can be attractive when capacity and protection against two disk failures are priorities.

Regardless of the RAID choice, maintain separate backups, verify restore procedures, monitor disk health, and document the replacement process. Redundancy improves availability but does not guarantee data safety.

Q7. Design a Memory-Management Plan from First Principles

1. Introduction

Memory management is an important operating-system function that allocates, protects, and monitors memory used by processes.

A smart factory edge device has 2 GB of RAM and no swap disk. It runs three tasks:

1. Real-time control task.

2. Machine-learning task requiring 1.5 GB.

3. Logging task.

The real-time task must not be delayed by page faults. Therefore, the memory-management plan must prioritise predictable execution.

2. Stage 1 – Requirements Analysis

Memory Timing

Task Page-fault tolerance

requirement requirement

Real-time Small and Cannot tolerate page faults

Strict deadline

control predictable during critical execution

Machine- Approximately 1.5 High memory May tolerate occasional

learning GB demand faults

Depends on log

Logging Lower priority Can tolerate some delay

volume

3. Measurable Requirements

The design should satisfy these requirements:

The real-time task must have its required memory allocated before critical execution.

Its critical memory pages must remain resident.
The machine-learning task must stay within its assigned memory budget.

Logging must not consume memory required by the control task.
The system must prevent uncontrolled memory allocation.
New tasks must be admitted only after checking available memory.

4. Stage 2 – Compare Memory-Management Mechanisms

The design begins by identifying the system's workload and constraints: total RAM, timing requirements, task priorities, peak memory needs, and consequences of delay. A plan must prevent memory exhaustion and keep time-critical tasks predictable.

Memory-management policy is not simply a choice of one algorithm. It includes allocation, protection, admission control, monitoring, and recovery. The most suitable combination is determined by the application's behaviour and reliability requirements.

Contiguous Allocation

Allocates one continuous block to a process.
Simple to understand.
Can suffer from external fragmentation.
Resizing a block may be difficult.
Paging

Divides memory into fixed-size pages.
Avoids external fragmentation.
Supports memory protection.
Page faults may cause unpredictable delays.
Segmentation

Divides a program into logical regions.
Supports different protection rules for code, stack, and data.
Can suffer from external fragmentation.
Demand Paging

Loads pages only when accessed.
Reduces initial memory requirements.
May cause page-fault delays.
Not suitable for a time-critical loop that cannot tolerate page faults.

5. Comparison Table

Mechanism Real-time control Machine-learning Logging

Predictable if

Contiguous May be difficult for Suitable for small

allocated in

allocation large memory fixed buffers

advance

Suitable with

Paging Suitable Suitable

locked pages

Less flexible for

Supports logical

Segmentation large dynamic Suitable

protection

memory

Contiguous allocation is simple but vulnerable to external fragmentation and difficult resizing. Paging is flexible and eliminates external fragmentation, while segmentation reflects logical program components and supports per-segment protection.

A practical design often uses paging as its basic mechanism and supplements it with virtual-memory policies. Critical tasks may require reserved or locked pages to avoid unpredictable page-fault delays.

Avoid for critical Useful for non- Acceptable when

Demand paging

execution critical data delay is allowed

6. Memory Allocation Plan

The 2 GB RAM should be divided using a reserved memory budget.

An illustrative plan is:

Component Memory budget

Real-time control task 128 MB

Machine-learning task 1,500 MB

Logging task 64 MB

Operating system and drivers 256 MB

Remaining reserve 100 MB

Total 2,048 MB

This is an example allocation plan. The actual allocation must be validated

against the real task requirements and operating-system overhead.

7. Memory Locking with mlock()

The mlock() system call locks a memory region into RAM so that the pages are

not swapped out.

For a time-critical task, the program can allocate its memory in advance and

lock the required pages.

C program:

```
    #include <stdio.h>
    #include <stdlib.h>
    #include <sys/mman.h>
#define SIZE 1024 * 1024
    int main()
    {
char *memory = malloc(SIZE);
```

A memory budget should be treated as an admission-control limit. The operating system and application should reserve enough memory for the real-time task, essential services, buffers, and safety margin before permitting optional workloads to run.

Demand paging is useful for noncritical tasks that can tolerate delay. It should be restricted or carefully controlled for tasks whose deadlines must be met, because page faults may involve storage I/O and take much longer than normal memory access.

```
    if (!memory) {
perror("malloc");
return 1;
    }
    if (mlock(memory, SIZE) != 0) {
perror("mlock");
free(memory);
return 1;
    }
    for (int i = 0; i < SIZE; i++)
memory[i] = 0;
    printf("Memory locked successfully\n");
munlock(memory, SIZE);
free(memory);
    return 0;
    }
```

Compilation:

gcc memory_lock.c -o memory_lock

8. Why Memory Locking Is Important

Prevents locked pages from being swapped out.
Reduces page-fault-related delays during critical execution.
Makes memory behaviour more predictable.
Helps time-critical applications meet deadlines.
The sample program demonstrates allocation, page locking, initialisation, and release. Every resource acquisition should have a corresponding cleanup path. Error handling is essential because allocation and locking can fail when the system is under memory pressure or limits are reached.

Locked memory reduces the risk that the selected region will be paged out, but it consumes a limited system resource. Applications should lock only the memory they truly need and should release it when the critical operation finishes.

However, mlock() alone does not guarantee real-time execution. CPU scheduling, interrupts, driver delays, and other kernel activities can still affect execution time.

9. Memory Simulation

The system can simulate the memory usage of all three tasks.

Task Example memory usage

Real-time control 128 MB

Machine-learning 1,500 MB

Logging 64 MB

Total task usage 1,692 MB

Remaining memory:

\[2048-1692=356MB \]

The remaining memory must also support the operating system, drivers, and other required services. The allocation plan should be adjusted if actual usage exceeds the budget.

10. Verification

The following tests can be performed:

1. Run the control task without memory locking.

2. Record execution-time measurements.

3. Run the control task with memory locking.

4. Record execution-time measurements again.

5. Compare average and worst observed delays.

6. Repeat the test under machine-learning and logging workloads.

Test Memory locking Observed worst-case delay

Test 1 Disabled Record measured result

Test 2 Enabled Record measured result

The values must come from actual execution. A single observed maximum is not proof of a guaranteed worst-case execution time.

11. Stage 4 – Fourth Task of 500 MB

If a fourth task requiring 500 MB is added:

Predictability also depends on CPU scheduling, interrupt handling, device drivers, and synchronisation. A memory plan should therefore be integrated with the broader real-time design rather than treated as a guarantee by itself.

Memory simulation can estimate whether the planned budgets fit within physical RAM. It should include peak usage, not only average usage, and should be repeated when task sizes or software versions change.

Total task memory demand increases.

The system may not have enough free memory.
The machine-learning task may need a smaller memory budget.
The new task should not be allowed to interfere with the real-time task.
Memory admission control must be performed before starting the new task.

The system may need to move the machine-learning task to a separate device or reduce its memory usage.

12. Alternative: Real-Time Operating System

A real-time operating system without virtual memory can provide more predictable memory behaviour.

RTOS without virtual

Feature Linux with locked memory

memory

Memory Virtual memory with selected Usually fixed physical-

management locked pages memory allocation

Avoided for locked pages

Page-fault delay No demand-paging faults

after preparation

Real-time Depends on configuration Often designed for

predictability and workload predictable timing

Machine-learning

Can use remaining memory Limited by fixed RAM

task

Requires careful memory

Adding new tasks More flexible

planning

13. Final Recommendation

The edge device should reserve memory for the real-time control task, lock its critical pages using mlock(), and prevent lower-priority tasks from consuming its memory budget.

The machine-learning task should use controlled memory allocation, while logging should use a limited buffer.

For applications with extremely strict timing requirements, a real-time

operating system may be more suitable than a general-purpose Linux

configuration.

The final policy should define what happens when memory demand grows beyond the budget: reject or defer optional work, reduce the machine-learning workload, preserve the real-time task, and log the event for later analysis. Silent overcommit is unsuitable for a safety-critical edge device.

Regular validation is required. Measure worst-case memory use, test allocation failures, check for leaks, and verify that the system remains stable under simultaneous task execution. These practices make the plan maintainable and auditable.

OPERATING SYSTEMS – ASSIGNMENT 3 (CO3)

PROGRAMS / COMMANDS DOCUMENT (Separate Coding Reference)

Note: Compile and execute on Linux. Record actual outputs, timings, memory measurements and screenshots in your report; do not claim measurements without running.

Q1 – Dynamic library and runtime loading

```
/* hello.c */

#include <stdio.h>

void hello(void) { puts("Hello from shared library"); }


/* bank.c */

#include <stdio.h>

int main(void) { puts("Banking Application"); return 0; }


/* runtime.c */

#include <stdio.h>

#include <dlfcn.h>

int main(void) {

    void *h = dlopen("./libhello.so", RTLD_LAZY);

    if (!h) { fprintf(stderr, "%s\n", dlerror()); return 1; }

    void (*hello)(void) = (void (*)(void)) dlsym(h, "hello");

    if (!hello) { fprintf(stderr, "%s\n", dlerror()); dlclose(h); return 1; }

    hello();

    dlclose(h);

    return 0;

}


/* Commands:

gcc -c hello.c -fPIC

gcc -shared -o libhello.so hello.o

gcc runtime.c -o runtime -ldl

gcc bank.c -o bank_static -static
```

```
gcc bank.c -o bank_dynamic

ls -l bank_static bank_dynamic

ldd bank_dynamic

./runtime

cat /proc/self/maps

/usr/bin/time -v ./bank_dynamic

*/
```

Q2 – Paging address translation (Complete the Code)

```
#include <stdio.h>

#define PAGE_SIZE 1024

#define PAGES 8

int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};


int main(void) {

    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};

    for (int i = 0; i < 6; i++) {

        unsigned a = logical[i], p = a / PAGE_SIZE, d = a % PAGE_SIZE;

        printf("%5u -> page %u offset %6u : ", a, p, d);

        if (p >= PAGES) puts("invalid address");

        else if (page_table[p] == -1) puts("page fault");

        else printf("physical %u\n", (unsigned)page_table[p] * PAGE_SIZE + d);

    }

    return 0;

}

/* Compile: gcc -std=c11 -Wall -Wextra paging.c -o paging

   Experiment: change PAGE_SIZE to 2048 and recalculate. */
```

Q3 – Segmentation translation and invalid access

```
#include <stdio.h>

int main(void) {

    int base[] = {1000, 3000, 5000};

    int limit[] = {500, 800, 400};

    int segment, offset;

    printf("Enter segment number: "); if (scanf("%d", &segment) != 1) return 1;

    printf("Enter offset: "); if (scanf("%d", &offset) != 1) return 1;

    if (segment < 0 || segment >= 3) puts("Invalid segment");

    else if (offset < 0 || offset >= limit[segment]) puts("Segmentation fault");

    else printf("Physical address = %d\n", base[segment] + offset);

    return 0;

}

/* Compile: gcc -std=c11 -Wall -Wextra segmentation.c -o segmentation

   Real SIGSEGV demonstration (run separately):

   int *p=NULL; *p=10; */
```

Q4 – Sequential/random memory access and page-fault observation

```
/* memtouch.c: allocate/touch memory; observe faults with /usr/bin/time -v */

#define _GNU_SOURCE

#include <stdio.h>

#include <stdlib.h>

#include <stdint.h>

#include <unistd.h>

int main(int argc, char **argv) {

    size_t mb = (argc > 1) ? strtoul(argv[1], NULL, 10) : 256;

    size_t bytes = mb * 1024 * 1024;

    long ps = sysconf(_SC_PAGESIZE);

    volatile unsigned char *p = malloc(bytes);

    if (!p) { perror("malloc"); return 1; }

    for (size_t i=0; i<bytes; i+=(size_t)ps) p[i] = 1; /* sequential */

    /* pseudo-random page order */

    size_t pages=bytes/(size_t)ps;

    for (size_t k=0;k<pages;k++) {

        size_t idx=(k*2654435761u)%pages;

        p[idx*(size_t)ps] += 1;

    }

    printf("Touched %zu MB; page size %ld\n", mb, ps);

    free((void*)p); return 0;

}

/* gcc -O2 memtouch.c -o memtouch

   /usr/bin/time -v ./memtouch 512

   COW observation: fork(), read getrusage(RUSAGE_SELF).ru_maxrss,

   then child writes one byte per page and compare RSS. */
```

Q5 – FIFO, LRU and Optimal page replacement (including TODO)

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page) {

    for (int i=0;i<n;i++) if (f[i]==page) return i;

    return -1;

}

int fifo(const int ref[], int len, int frames) {

    int f[MAXF], used=0, next=0, faults=0;

    for(int i=0;i<len;i++) {

        if(find(f,used,ref[i])>=0) continue;

        faults++;

        if(used<frames) f[used++]=ref[i];

        else { f[next]=ref[i]; next=(next+1)%frames; }

    } return faults;

}

int lru(const int ref[], int len, int frames) {

    int f[MAXF], last[MAXF], used=0, faults=0;

    for(int i=0;i<len;i++) {

        int k=find(f,used,ref[i]);

        if(k>=0) { last[k]=i; continue; }

        faults++;

        if(used<frames) { f[used]=ref[i]; last[used]=i; used++; }

        else {

            int victim=0;

            for(int j=1;j<frames;j++)

                if(last[j]<last[victim]) victim=j;

            f[victim]=ref[i]; last[victim]=i;

        }

    } return faults;

}
```

```
int optimal(const int ref[], int len, int frames) {
    int f[MAXF], used=0, faults=0;
    for(int i=0;i<len;i++) {
        if(find(f,used,ref[i])>=0) continue;
        faults++;
        if(used<frames) { f[used++]=ref[i]; continue; }
        int victim=0, farthest=-1;
        for(int j=0;j<frames;j++) {
            int next=len;
            for(int t=i+1;t<len;t++) if(ref[t]==f[j]) { next=t; break; }
            if(next>farthest) { farthest=next; victim=j; }
        } f[victim]=ref[i];
    } return faults;
}
int main(void) {
    int r[]={7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    for(int fr=3;fr<=4;fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n",fr,
        fifo(r,20,fr),lru(r,20,fr),optimal(r,20,fr));
    int b[]={1,2,3,4,1,2,5,1,2,3,4,5};
    printf("Belady string FIFO: 3=%d, 4=%d\n",fifo(b,12,3),fifo(b,12,4));
    return 0;
}
/* gcc -std=c11 -Wall -Wextra replacement.c -o replacement */
```

Q6 – Linux disk and swap inspection (commands)

```
# View block devices, partitions and mount points

lsblk

sudo fdisk -l


# View RAM and swap

free -h

swapon --show

cat /proc/swaps


# Read speed (use a test file; do not benchmark live patient data)

dd if=/path/to/testfile of=/dev/null bs=1M status=progress

# For safer cache-aware measurement, use a disposable test file and document

# cache conditions. Never run destructive write tests on production disks.


# RAID capacity, 6 x 4 TB (decimal):

# RAID 0 = 24 TB; RAID 5 = (6-1)*4 = 20 TB;

# RAID 6 = (6-2)*4 = 16 TB; RAID 10 = 12 TB (3 mirrored pairs).
```

Q7 – Lock memory for a critical loop using mlock()

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>
volatile unsigned long counter;
int main(void) {
    size_t n=1024*1024;
    char *buf=malloc(n);
    if(!buf) { perror("malloc"); return 1; }
    /* Touch every page before timing; then lock resident memory. */
    long ps=sysconf(_SC_PAGESIZE);
    for(size_t i=0;i<n;i+=(size_t)ps) buf[i]=0;
    if(mlock(buf,n)!=0) { perror("mlock (check privileges/RLIMIT_MEMLOCK)"); }
    struct timespec a,b;
    clock_gettime(CLOCK_MONOTONIC,&a);
    for(unsigned long i=0;i<1000000;i++) counter++;
    clock_gettime(CLOCK_MONOTONIC,&b);
    double ms=(b.tv_sec-a.tv_sec)*1000.0+(b.tv_nsec-a.tv_nsec)/1e6;
    printf("Loop duration: %.3f ms\n",ms);
    munlock(buf,n); free(buf); return 0;
}
/* gcc -std=c11 -Wall -Wextra rtlock.c -o rtlock
   Run repeatedly and report max observed delay. This is not a hard real-time
   guarantee on general-purpose Linux; measure under controlled load. */
```